CET257 Enterprise Project: Assignment 2

Task B4: Report on the Security of the Application

Team Speed (Team 6) · Lift Right: 360° Manual Handling Training Simulation

1. Scope and context

Lift Right is a browser-based training simulation. Workers log in, complete six safety checkpoints and receive a score. Supervisors use an Admin Command Center to see worker results and export them to CSV. The prototype is a single static web page with **no server**: accounts, attempts and statistics are held in the browser's local storage. This report covers what we did for security during development, the weaknesses we found when we reviewed our own code, and what must change before real use. We reviewed the source directly, so the findings below are evidence-based and not assumptions.

2. Security considered during development

- **Small attack surface:** with no back end or database, no worker data leaves the device, and there is no server to attack.
- **Data minimisation:** we store only a username, a hashed password, role, attempts, times and scores. We do not collect email addresses, dates of birth or other personal details.
- **Role separation:** worker and administrator accounts are distinct, and administrator sign-up needs a registration code.
- **Input handling:** usernames are trimmed, lower-cased, stripped of spaces, slashes and quotes, and capped at 60 characters. CSV cells are wrapped in quotes with embedded quotes escaped.
- **Passwords are not stored in readable form** and the sign-in form lets users show or hide what they type.
- **Controlled deletion:** the admin can remove individual workers or clear worker data, and administrator accounts are kept when worker data is cleared.
- **Transport:** the live prototype is served over HTTPS by its host.

3. Weaknesses found in our own prototype

Weakness	Evidence in the code	Impact	Priority
Role check can be bypassed	Choosing Administrator on the log-in form with any valid worker account opens the Command Center. We confirmed this with an automated test on 11 October 2026	Any worker can see and delete every worker's records	High
Weak password hashing	Passwords use a short, unsalted 32-bit hash, which can be reversed quickly	Stolen records expose passwords	High
Client-side-only authentication	The administrator check is a variable in the page and accounts sit in editable local storage	Anyone can bypass admin login or edit scores	High

Weakness	Evidence in the code	Impact	Priority
Shared admin secrets in the source	The admin registration code is written in the script, and a demo credential is in a code comment	Anyone viewing the source can become an admin	High
Incomplete output encoding	Usernames are placed into the page with innerHTML, and angle brackets are not encoded	Script injection if the filtering is bypassed	Medium
Third-party scripts	Three.js and fonts load from public CDNs with no integrity check or content security policy	A compromised CDN could run code in our page	Medium
CSV formula injection	Exported cells are quoted but not neutralised if they start with =, +, - or @. Confirmed by an automated test	Malicious names could run formulas in Excel	Medium
No brute-force or session limits	We found no login attempt limit, timeout or lock-out	Guessing and unattended sessions	Medium

4. The client's perspective

Training records about named employees are personal data under UK GDPR and the Data Protection Act 2018 (ICO, no date). A real deployment would need a lawful basis, a retention period, restricted supervisor access and a way to answer access and deletion requests. A bigger business risk is trust: because scores sit in editable browser storage, a worker could change their own result, so the records could not be relied on as compliance evidence for an audit or insurer. The client also depends on availability. Records held on a single PC are lost if the browser data is cleared.

5. The end user's perspective

Warehouse staff often share terminals, so an unattended session could expose another person's results. Workers need to know who can see their scores, and that the password they choose should not be reused elsewhere. Training should never be capable of unfairly marking someone down because of a technical fault, so the system needs clear error messages and a way to resume an interrupted attempt.

6. Future security measures

- **Move authentication to a server:** hash passwords with Argon2id or bcrypt with a unique salt, issue session tokens in HttpOnly, Secure, SameSite cookies, and check roles on the server for every request.
- **Replace the shared admin code** with invitation links that expire, and add multi-factor sign-in for administrators. As an immediate fix, the log-in must compare the selected account type with the stored role and refuse a mismatch.
- **Protect the page:** add a Content Security Policy, host Three.js and fonts ourselves or use subresource integrity, and use textContent instead of innerHTML.
- **Harden exports and input:** prefix risky CSV cells, validate all fields on the server, and add login rate limiting, lock-out and an idle timeout.

- **Protect the data:** use an encrypted database with backups, an audit log of administrator actions, a retention policy and a data protection impact assessment.
- **Test before launch:** run a penetration test against the OWASP Top 10 (OWASP Foundation, 2021) and re-test after each release.

7. Conclusion

The prototype was designed to demonstrate training and scoring, not to protect real people's data, and its security is demonstration-level. We have identified each weakness, explained its effect on the client and on workers, and set out a clear route to fix it. The first priority is a server-side back end with proper authentication.

References

Information Commissioner's Office (no date) UK GDPR guidance and resources. Available at: https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/ (Accessed: 11 October 2026).

OWASP Foundation (2021) OWASP Top 10:2021. Available at: https://owasp.org/Top10/ (Accessed: 11 October 2026).